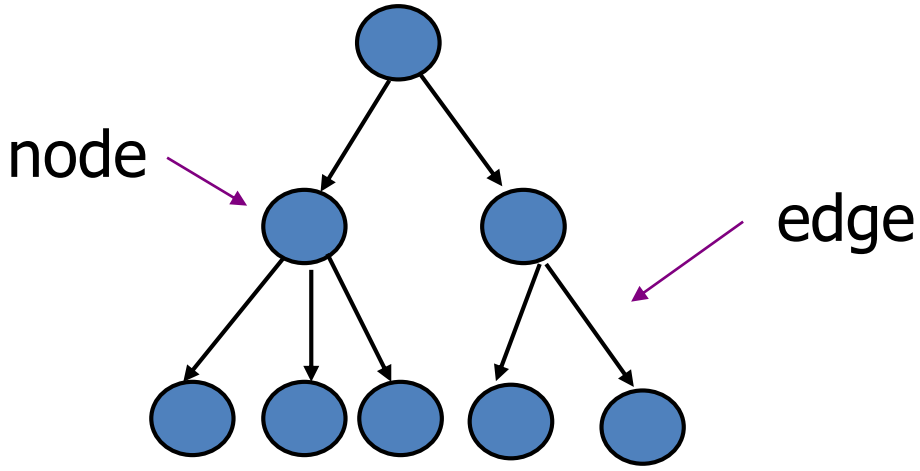
Tree Structure

What is a Tree?

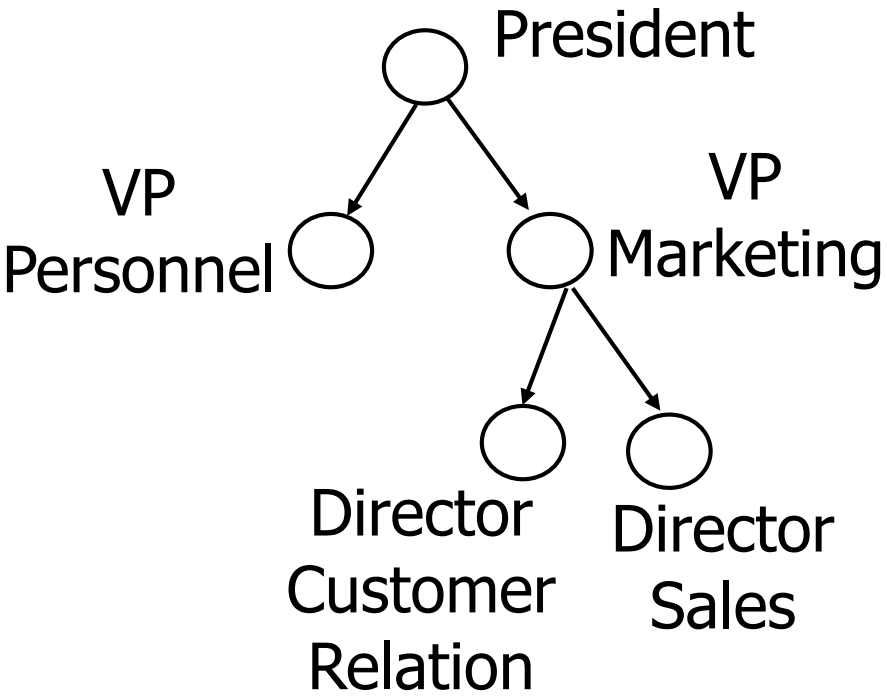
A connected acyclic graph is a Tree.

- Each tree consists of nodes and edges
- Each node represents an object
- Each edge represents the relationship between two nodes.
- Trees are structures used to represent hierarchical relationship

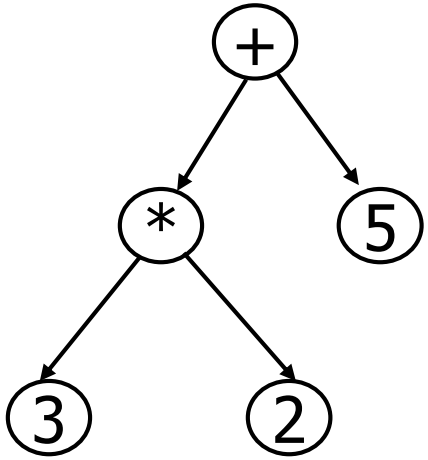


Some applications of Tree

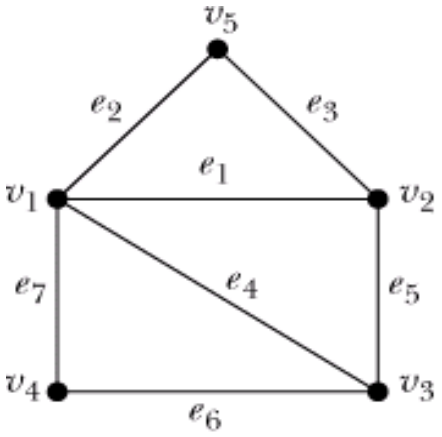
Organization Chart

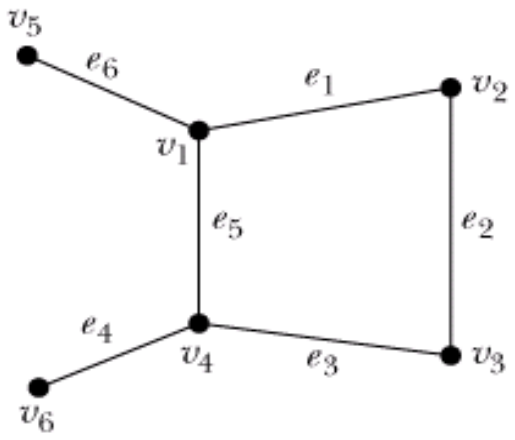


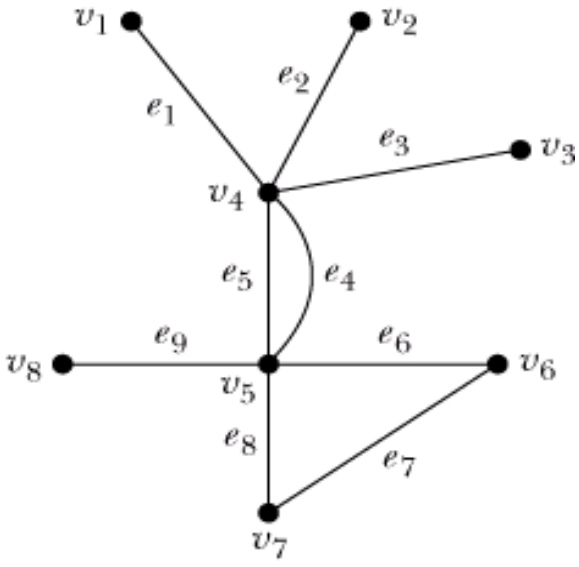
Expression Tree



Not Tree





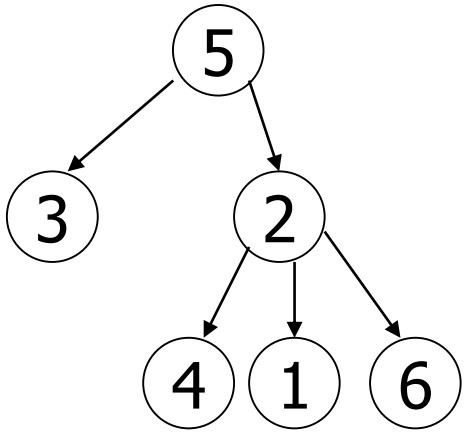


Each of these graphs has a cycle. Hence, none of these graphs is a tree.

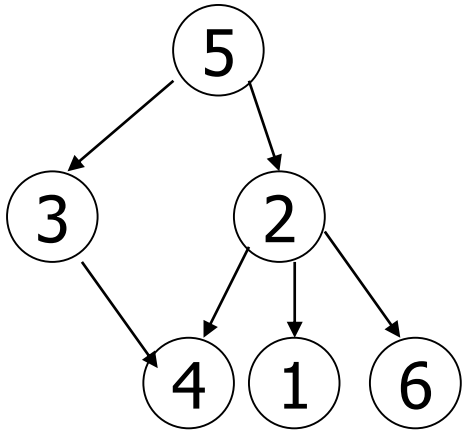
Properties of a Tree

A Tree satisfies the following properties:

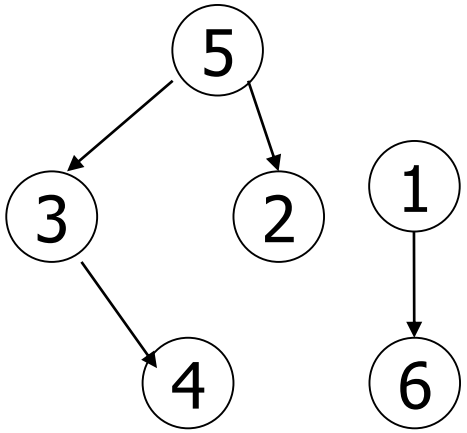
1. It has one designated node, called the root, that has no parent.
2. Every node, except the root, has exactly one parent.
3. A node may have zero or more children.
4. There is a unique directed path from the root to each node.



tree



Not a tree

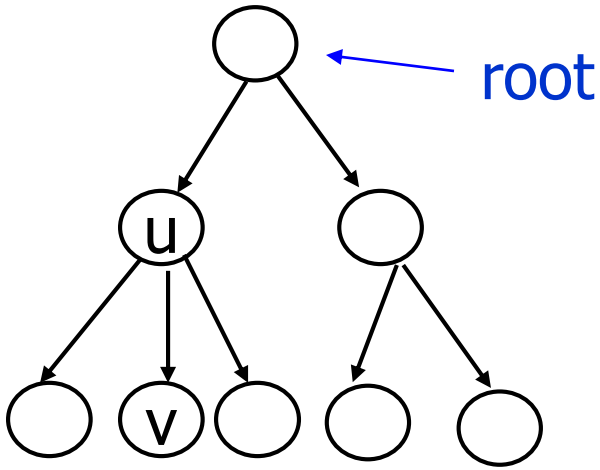


Not a tree

Rooted tree

A rooted tree is a tree in which one vertex has been designated as root and every edge is directed away from the root.

A tree together with its root produces a directed graph.



Tree is a Graph

Let T be a graph with n vertices. Then the following conditions are equivalent.

(i) T is a tree.

(ii) If u and v are two vertices in T, then there exists only one path from u to v.

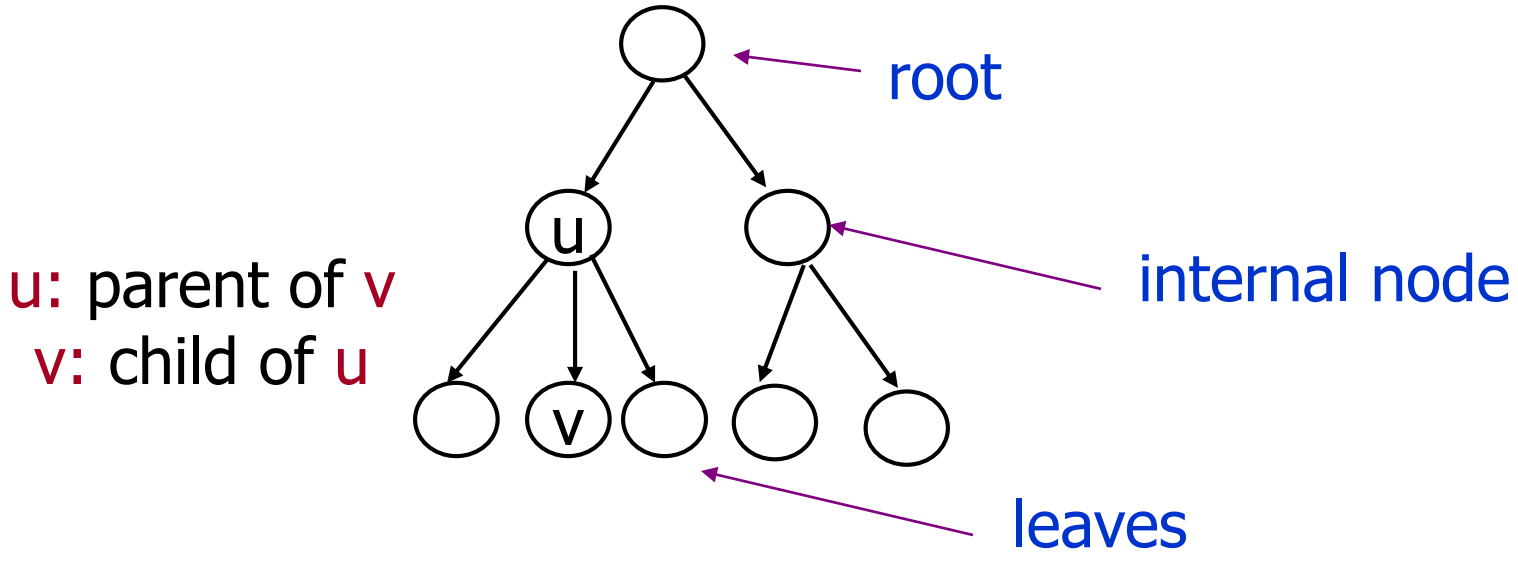
(iii) T is a connected graph and has $n - 1$ edges.

(iv) T has no cycles and has $n - 1$ edges.

Terminology

- For any two nodes u and v, if there is an edge from u to v, u is called the parent of v while v is called the child of u. Such edge is, typically, denoted as (u, v).

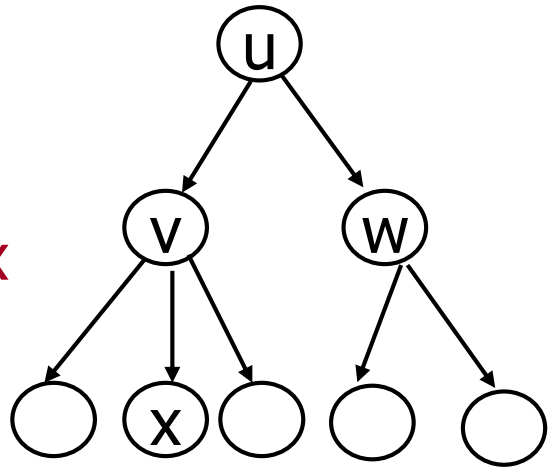
- In a tree, there is exactly one node without parent, which is called the root. The nodes without children are called leaves.



Terminology

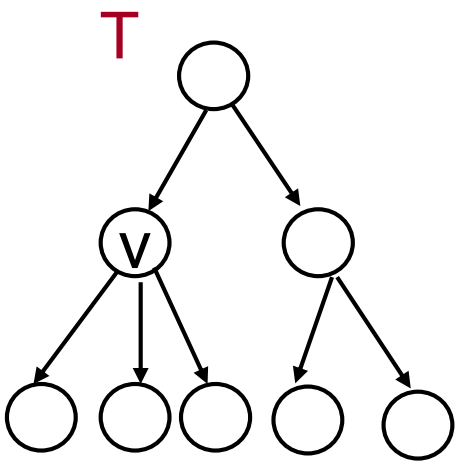
- If two nodes have the same parent, they are siblings.
- A node u is a predecessor of v if u is parent of v or parent of parent of v or …
- A node v is a successor of u if v is child of u or child of child of u or …

v and w are siblings
u and v are predecessors of x
v and x are successors of u

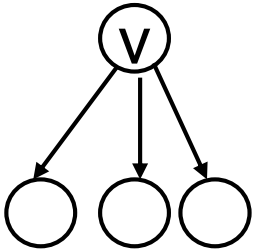


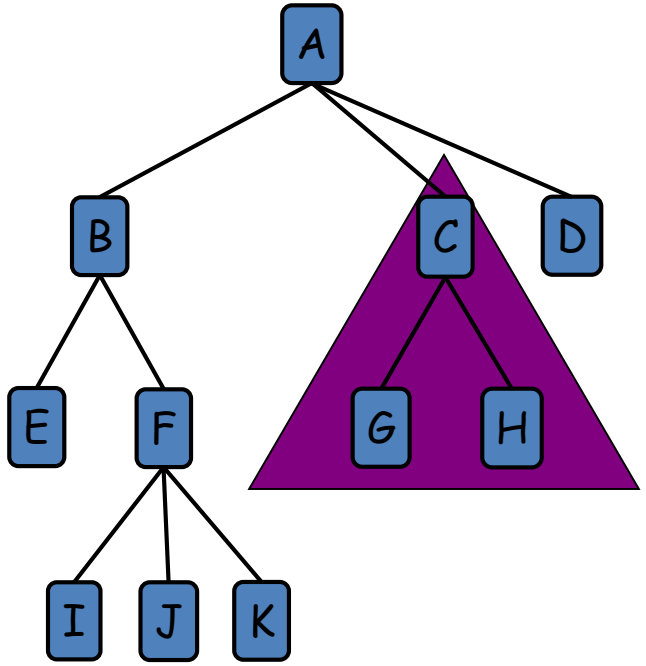
Terminology

A subtree is any node together with all its successors.



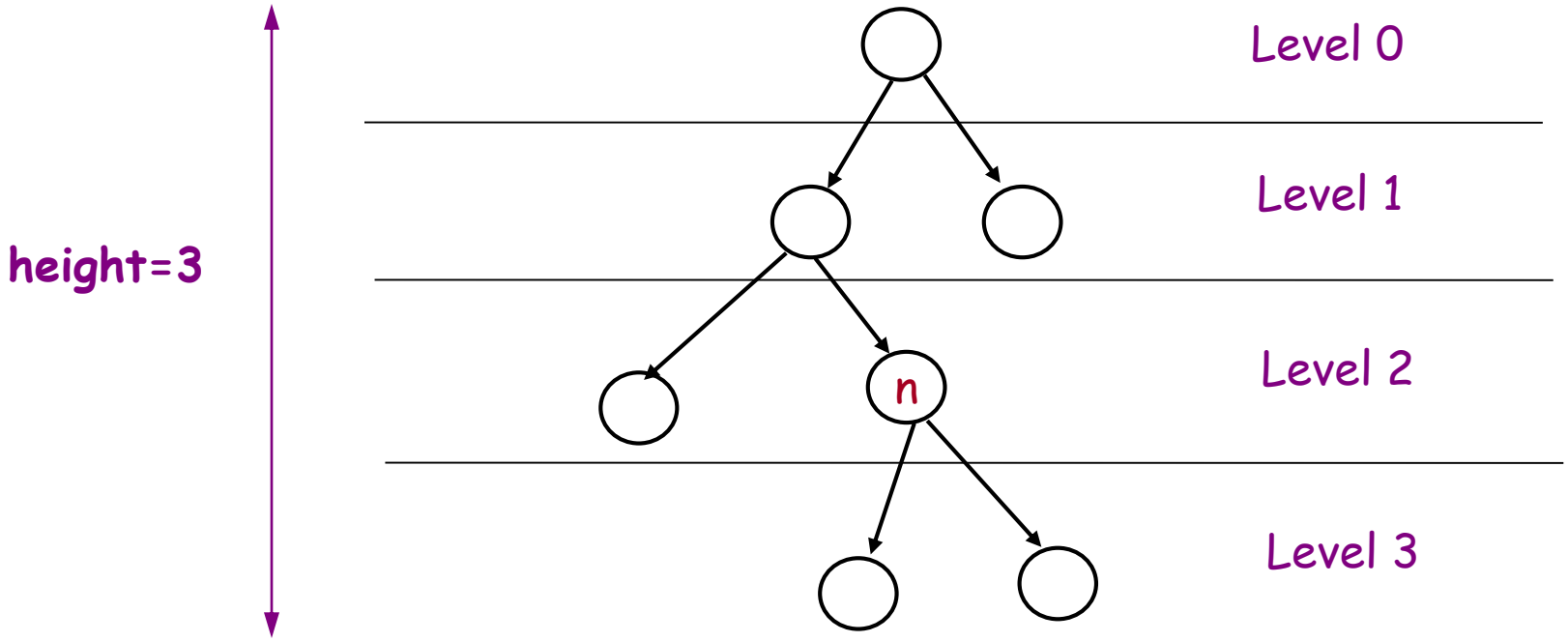
subtree of T





Terminology

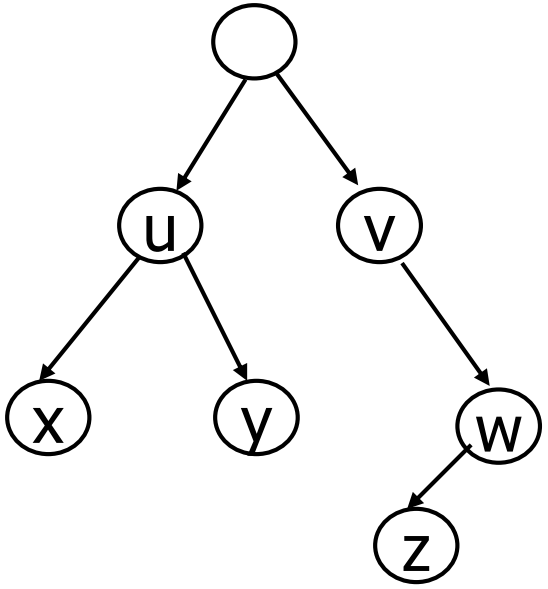
- Level of a node n: the length of the path from root to node n
- Height of a tree: maximum level among all of its node



Binary Tree

A rooted tree is called an m-ary tree if every node (except leaf) has no more than m children.

Binary Tree is a tree in which every node (except leaf) has at most 2 children.

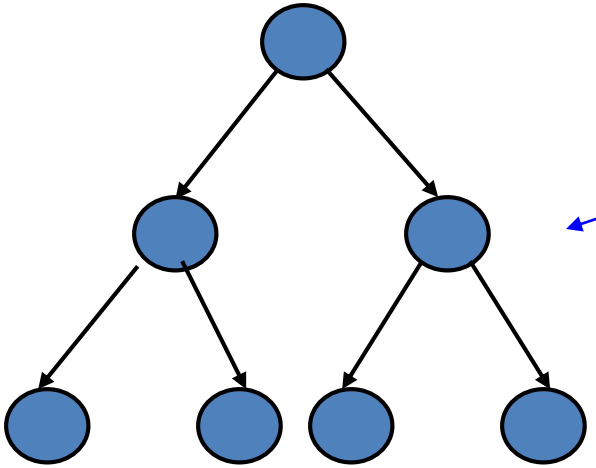


x: left child of u
y: right child of u
w: right child of v
z: left child of w

Full Binary tree

A full m-ary tree is a tree in which each internal node has exactly m children.

A full binary tree is a binary tree in which each internal node has exactly two children.



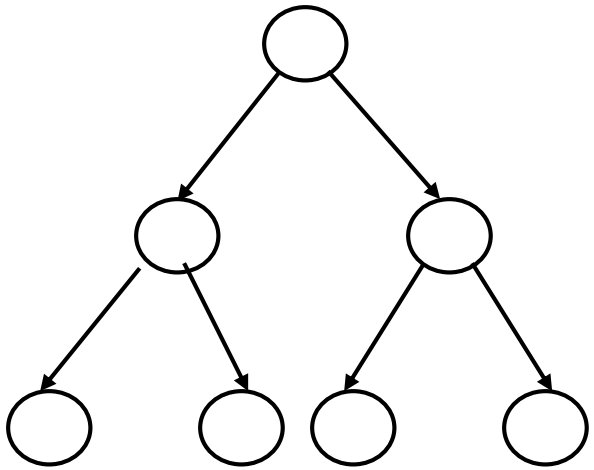
Full binary tree of height 2

Property of a full Binary tree

A full binary tree of height h has $2^{h+1} - 1$ nodes

No. of nodes $= 2^0 + 2^1 + \ldots + 2^h$

$= 2^{h+1} - 1$



Level 0: 2^0 nodes

Level 1: 2^1 nodes

Level 2: 2^2 nodes

Property of a Binary tree

Consider a binary tree T of height h. The number of nodes of $T \leq 2^{h+1} - 1$

Reason: you cannot have more nodes than a full binary tree of height h.

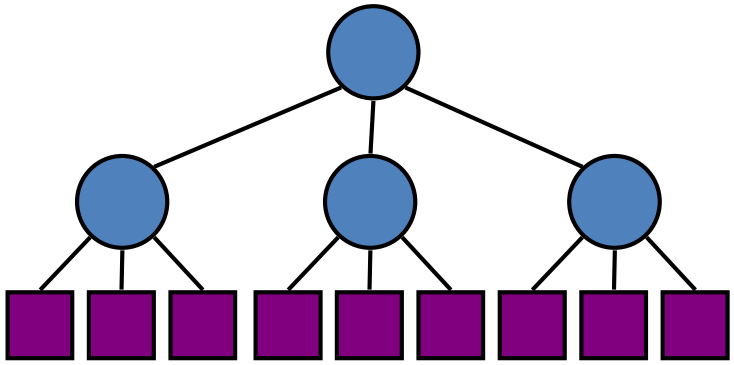
Property of a Binary tree

Prove that the height of a binary tree with n nodes is $O(\log n)$

Divide-and-Conquer

Divide-and conquer is a general purpose problem solving approach:

- Divide: divide the input data S into two or more disjoint subsets S_1, S_2, …
- Recur: solve the subproblems recursively
- Conquer: combine the solutions for S_1, S_2, …, into a solution for S



Binary search

Logic:

1) Compare the target item to the middle positioned value in the array
   - If the middle value is larger, then the search can be limited to the left half of the array
   - If the middle value is smaller, then the search can be limited to the right half of the array

2) Apply above logic to array of size n/2, and so on.

3) Finally, either the item will be found or else the interval will become empty, implying that the item does not occur in the array.

Binary Search in Ordered Array

Input: An ascending array of elements **A[1],…,A[n] ,** target x.

Output: the index of the first occurrence of x in array (0 if it doesn't exist)

Algorithm binarySearch(A, n, x):

```
l =1, r = n
while (l <= r) {
        mid = ⌊ (l + r) / 2⌋
        if A[mid] = x                   // found item
            return mid
        if x < A[mid]
            r = mid -1                  // search left half
        else l = mid +1                 // search right half
}
return 0
```

Time complexity

- Steps 2, 4 and 5 require exactly one comparison
- In the worst case, each iteration of the loop uses exactly 3 comparisons

```
binarySearch(A, n, x):
1. l =1, r = n
2. while (l <= r) {
3.    mid = ⌊ (l + r) / 2⌋
4.    if A[mid] = x
            return mid
5.    if x < A[mid]
            r = mid -1
      else l = mid+1
}
6. return 0
```

Time complexity (contd.)

- Suppose *k* is the number of times that *n* can be divided in half until the result is less than or equal to 1
- Then $n/2^k = 1$
  $\Rightarrow k = \log_2 n$
- Therefore, time complexity = O(log n)

```
binarySearch(A, n, x):
1. l =1, r = n
2. while (l <= r) {
3.      mid = ⌊ (l + r) / 2⌋
4.     if A[mid] = x
                return mid
5.     if x < A[mid]
                r = mid -1
       else  l = mid+1
   }
6. return 0
```